THE COMPLETE HANDBOOK

OPERATING SYSTEMS MASTERY

From Fundamentals to Advanced Systems, Linux & Interviews

Beginner · Intermediate · Advanced · Interview Mastery

Processes · Threads · Scheduling · Synchronization · Deadlocks
Memory · Virtual Memory · File Systems · I/O · IPC · Storage
Networking · Linux Internals · Virtualization · Containers
100+ Interview Questions · Solved Numericals · 16 Projects

A SELF-CONTAINED LEARNING COURSE IN BOOK FORM

How to Use This Book

This is not a summary and not a list of topics. It is a complete, self-contained course that assumes you know almost nothing about operating systems and takes you all the way to the point where you can explain OS internals, solve numerical problems, write systems code, and answer senior-level interview questions with confidence.

You do not need another textbook alongside this one. Every concept is built from intuition first, then formalized with correct technical detail, then connected to real Linux behavior, real code, and the questions you will actually be asked. Read it front to back the first time — the order is deliberate, and later parts assume the earlier ones.

The Learning Path

Everything is organized along a single rising path. You will feel each part get a little deeper than the last, but never jump a step.



How Each Concept Is Taught

Important concepts are taught with the same repeatable structure so your brain learns how to learn them. You will not see all twelve labels spelled out every time — that would be noise — but every major idea moves through this arc:

Step	What it gives you
1. What is it?	A one-line, jargon-free definition.
2. Why do we need it?	The problem it solves — motivation before mechanism.
3. Simple intuition	An everyday analogy you already understand.
4. Technical explanation	The precise, correct definition with proper terms.
5. How it works internally	What the hardware and kernel actually do.
6. Example	A concrete, worked case — often in C or on the Linux shell.
7. Diagram	A picture that shows the real mechanism, not decoration.
8. Real-world example	Where this shows up in Linux, servers, or your laptop.
9. Related concepts	Links to ideas you have met or will meet.
10. Common mistakes	The traps that cost marks and interview offers.
11. Interview questions	How it is actually asked, with model answers.
12. Quick revision	The compressed version for the night before.

The Boxes — and What They Mean

Throughout the book, colored boxes flag the five things your eye should be able to find in seconds when revising:

> ▣ **KEY CONCEPT**
>
> The core idea, stated precisely. If you remember nothing else from a section, remember what is in these boxes.

> ⚑ **INTERVIEW TIP**
>
> Exactly how to phrase an answer, what the interviewer is really testing, and the follow-up they will ask next.

> ⚠ **COMMON MISTAKE**
>
> A misconception almost everyone holds at first. Catching these is what separates a memorized answer from an understood one.

> ⚑ **REAL WORLD**
>
> Where the concept lives in an actual system — Linux, a database, a web server, your own machine.

> ↻ **QUICK REVISION**
>
> The compressed, high-density recap. Read these to warm up, and again the night before an exam or interview.

Other Elements

Code blocks
Real, compilable C/POSIX and real Linux commands in monospace. Type them out — you learn systems by running things, not by reading about them.

Comparison tables
The exact "X vs Y" distinctions interviewers love. Part XX collects them all.

Solved numericals
Scheduling, paging, page replacement, Banker's, and disk problems worked step by step, with every calculation shown.

Check Your Understanding
End-of-part checkpoints: conceptual questions, application questions, interview questions, a practical experiment, and a clear "you have mastered this when…" bar.

> ⓘ **A NOTE ON DEPTH VS. SIMPLICITY**
>
> Where a simple explanation and a fully precise one differ, this book gives you the simple one first so you understand, then the precise one so you are correct. It never trades away technical accuracy for comfort — it just orders them so the accuracy actually lands.

The Fastest Way to Get Value

1. **First pass:** read a part fully, do its checkpoint, run its practical task.
2. **Second pass:** read only the **Key Concept** and **Quick Revision** boxes.
3. **Interview prep:** Part XVIII (question bank) + Part XX (comparison tables) + Part XXIII (final revision).
4. **Exam prep:** Part XIX (numericals) + each part's checkpoint + Part XXIII cheat sheet.

Let's begin.

Table of Contents

FRONT MATTER

How to Use This Book 2

FOUNDATIONS

Part I — OS Fundamentals & Architecture 7
Part II — Processes 20
Part III — Threads & Concurrency 31
Part IV — CPU Scheduling 39

COORDINATION & MEMORY

Part V — Process Synchronization . . . . .
Part VI — Deadlocks . . . . .
Part VII — Memory Management . . . . .
Part VIII — Virtual Memory . . . . .

SYSTEMS & DEVICES

Part IX — File Systems . . . . .
Part X — I/O Systems . . . . .
Part XI — Inter-Process Communication . . . . .
Part XII — Storage Systems . . . . .
Part XIII — Networking from an OS Perspective . . . . .

PRACTICAL & ADVANCED

Part XIV — Linux Practical OS . . . . .
Part XV — Advanced Operating Systems . . . . .
Part XVI — What Happens When... . . . . .
Part XVII — Practical Projects . . . . .

MASTERY & REVISION

Part XVIII — Interview Mastery (100+ Questions) . . . . .
Part XIX — Numerical Practice . . . . .
Part XX — Comparison Tables . . . . .
Part XXI — Visual Learning Index . . . . .
Part XXII — Mastery Checkpoints . . . . .
Part XXIII — Final Revision & Mastery Test . . . . .

PART I

OS Fundamentals & Architecture

Before processes, memory, or scheduling make sense, you need to know what an operating system is, why a computer is almost useless without one, and how a single program is kept from crashing the whole machine.

In this part: What an OS is · Resource manager & extended machine · Kernel and user space · User vs kernel mode · Privileged instructions · System calls · Traps, interrupts, exceptions · OS services · Protection & security · The boot process · Monolithic, micro, hybrid & modular kernels

PART I

OS Fundamentals & Architecture

A modern computer is a pile of extraordinarily fast but extraordinarily literal hardware. The operating system is the layer of software that turns that raw hardware into something a human — and a program — can actually use safely. This part builds the mental model that every later part depends on.

1.1 What Is an Operating System?

What is it? An **operating system (OS)** is the master program that sits between your applications and the physical hardware. It controls the hardware and provides services that every program can use, so that programs never have to talk to the raw metal directly.

Why do we need it? Imagine writing a program that plays a sound. Without an OS you would have to know the exact model of sound card, write the precise byte sequences it expects, manage the electrical timing, and make sure no other program touches the card at the same time. Now do that again for every keyboard, disk, network card, and GPU ever made. It is hopeless. The OS solves this once, for everyone, and hands you a simple request like "play these bytes."

> ▣ **KEY CONCEPT**
>
> An operating system does two big jobs at once: it is a **resource manager** (it decides who gets the CPU, memory, and devices, and when) and an **extended machine / abstraction layer** (it hides ugly hardware details behind clean, uniform interfaces like files and processes).

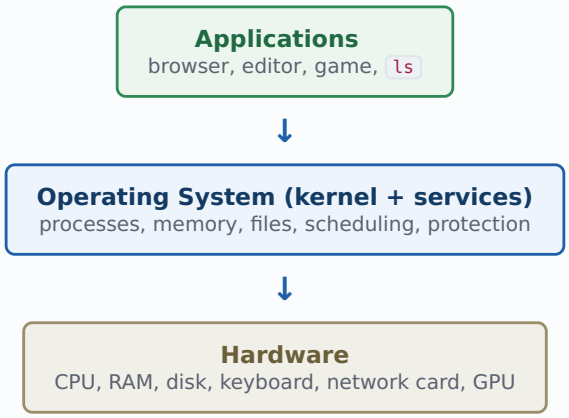
Simple intuition. Think of an OS as the manager of a busy shared workshop. Many workers (programs) want the same tools (CPU, memory, printer). The manager hands out tools fairly, stops two workers from grabbing the same tool at once, cleans up when a worker leaves, and gives everyone the same simple checkout process instead of making each worker learn how every tool is wired internally.

Technical explanation. The OS is a collection of software — centered on the **kernel** — that (1) manages hardware resources (processor time, memory, storage, I/O devices), (2) provides an execution environment for programs called the **process**, (3) exposes services through **system calls**, and (4) enforces **protection** so a fault or attack in one program cannot corrupt another or the OS itself.

How it works internally. The OS is loaded into memory at boot and stays resident. It runs in a privileged CPU mode so it can execute instructions ordinary programs cannot. When a program needs something only the OS can do (read a file, get more memory, send a packet), it makes a controlled request that switches the CPU into the

OS, the OS does the work, and control returns to the program. Between those requests, the OS is mostly idle — it is event-driven, woken by system calls and hardware interrupts.

Example. When you run `ls` in a terminal, the OS finds the program on disk, loads it into memory, gives it CPU time, lets it ask the file system for directory contents, sends its output to your terminal, and reclaims its memory when it exits — all without `ls` knowing anything about your specific SSD or display.



The OS is the mandatory middle layer: applications go through it to reach hardware.

Real-world example. Linux, Windows, macOS, Android, and iOS are operating systems. The same Chrome browser binary logic runs on wildly different laptops because the OS hides those differences. Your phone runs dozens of apps on one CPU because the OS shares it among them.

Related concepts. Kernel (§1.4), system calls (§1.7), processes (Part II), and protection (§1.10) are all facets of these two core jobs.

> ⚠ **COMMON MISTAKE**
>
> "The OS is the desktop, windows, and icons." That graphical shell is just one application running on top of the OS. You can run Linux with no graphics at all. The OS is the kernel and its services — the part that manages the hardware — not the wallpaper.

> ⚑ **INTERVIEW TIP**
>
> Asked "What is an operating system?", give the two-job answer in one breath: "It's the software that manages hardware resources and provides an abstraction layer so programs can use the hardware safely and portably through a uniform interface." Then be ready to expand either half.

1.2 Resource Manager & Extended Machine

These are the two classic lenses for understanding everything an OS does. Every feature in this book is one or the other (usually both).

The OS as a Resource Manager

The hardware has finite resources; many programs want them simultaneously. The OS multiplexes them — in **time** (the CPU runs process A for a few milliseconds, then B, then C, so fast it looks simultaneous) and in **space** (RAM is divided so each process gets its own region). It must do this fairly (no program starves), efficiently (little overhead), and safely (one program cannot read or wreck another's resources).

Resource	How the OS manages it	Covered in
CPU time	Scheduling: decides which process/thread runs next and for how long	Part IV
Main memory	Allocation, paging, virtual memory, protection	Parts VII–VIII
Storage	File systems, disk scheduling, free-space management	Parts IX, XII
I/O devices	Drivers, interrupts, DMA, buffering	Part X
Access / identity	Users, permissions, isolation	§1.10

The OS as an Extended (Virtual) Machine

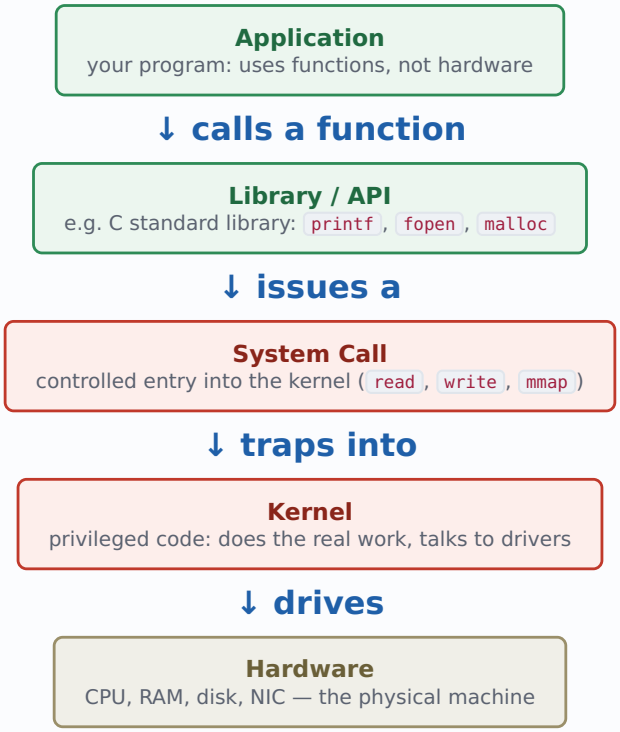
Raw hardware is hostile to program: disks store numbered blocks, not "files"; memory is one flat array shared by all; the CPU has no notion of "a program." The OS presents a friendlier **extended machine** where those hard things become easy abstractions:

Ugly hardware reality	Clean OS abstraction
Numbered disk sectors, no names	**Files** and directories with names
One shared physical RAM	A private **virtual address space** per program
One CPU, one instruction stream	Many concurrent **processes** and **threads**
Device-specific register pokes	Uniform `read()` / `write()` calls

> ▣ **KEY CONCEPT**
>
> **Abstraction** = hiding complexity behind a simpler interface. **Virtualization** = giving each program the illusion it has the whole machine to itself. The OS does both constantly: a file is an abstraction of disk blocks; a process is a virtualization of the CPU; virtual memory is a virtualization of RAM.

1.3 Hardware, OS, and Application: The Layered View

The single most useful picture in this book. Memorize it — nearly every question about "what happens when…" is really a question about how a request travels down and back up these layers.



The canonical stack. A request flows down; results (and interrupts) flow back up.

The two horizontal lines that matter most are: the **API boundary** (between your app and the library) and the **system-call boundary** (between user space and the kernel). The second is the famous "user/kernel line," and crossing it is not free — see §1.7.

1.4 Kernel, User Space, and Kernel Space

What is the kernel? The **kernel** is the core of the OS — the part that is always in memory, runs with full hardware privilege, and directly manages processes, memory, and devices. When people say "the OS did X," they usually mean the kernel did X.

Memory is conceptually split into two territories:

Kernel space
: The memory region where the kernel's code and data live. Only privileged (kernel-mode) code may access it. Protected from ordinary programs.

User space

Where ordinary programs (and their libraries) run. Each process gets its own user-space view. Code here cannot touch kernel space or other processes' memory directly.

> ▣ **KEY CONCEPT**
>
> **User space vs kernel space** is about memory and privilege territory. **User mode vs kernel mode** (next section) is about the CPU's current privilege state. They move together: code in user space runs in user mode; the kernel runs in kernel mode. Do not confuse the two axes — one is "where the memory is," the other is "what the CPU is allowed to do right now."

1.5 Dual-Mode Operation: User Mode and Kernel Mode

Why do we need it? If every program could run any instruction and touch any memory, one buggy or malicious program could halt the CPU, erase the disk, or read your passwords out of another program. To prevent that, the CPU itself supports (at least) two privilege levels.

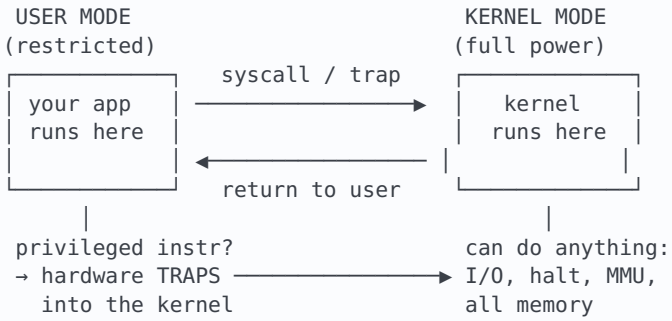
Kernel mode (supervisor / privileged mode)

The CPU can execute every instruction and access all memory and devices. The kernel runs here.

User mode

A restricted state. Privileged instructions are forbidden; memory access is limited to the process's own space. Applications run here.

How it works internally. A bit in a CPU register (the "mode bit," e.g. current privilege level / ring on x86) records the mode. In user mode, if a program attempts a privileged instruction or touches memory it does not own, the hardware immediately raises an **exception (trap)** and jumps into the kernel, which typically kills the offending program (the classic segmentation fault). The only legitimate ways to get from user mode into kernel mode are a **system call**, an **interrupt**, or an **exception** — all of which enter the kernel at fixed, trusted entry points. A program can never simply "switch itself" to kernel mode; that is the whole point.



Dual-mode operation. The mode bit gates what the CPU may do; controlled doorways cross it.

> ⚠ **COMMON MISTAKE**
>
> Thinking a program "calls the kernel like a normal function." It does not — a normal call cannot change privilege. A system call executes a special CPU instruction (e.g. `syscall` on x86-64) that atomically switches to kernel mode and jumps to a fixed kernel entry point. That hardware-enforced doorway is what makes the boundary safe.

1.6 Privileged Instructions

A **privileged instruction** is a machine instruction that can only be executed in kernel mode. These are exactly the operations that could compromise the whole system if any program could run them:

- Halting the CPU, disabling/enabling interrupts.
- Loading the page-table base register / switching address spaces (controlling the MMU).
- Direct access to I/O device registers.
- Modifying the mode bit or system control registers.
- Executing `hlt`, setting timers, flushing the TLB.

Attempting any of these in user mode raises a trap. This single mechanism — "some instructions are off-limits unless you are the kernel" — is the hardware foundation of all OS protection.

1.7 System Calls — and API vs System Call

What is it? A **system call** is the controlled mechanism by which a user-mode program requests a service from the kernel. It is the only door through which a program can ask the OS to do something privileged (open a file, allocate memory, create a process, send data on the network).

Simple intuition. You (a program) cannot walk into the bank vault (hardware). You slide a request form through a small, guarded window (the system call). A teller (the kernel) verifies your request, does it on your behalf, and slides the result back. You never touch the vault yourself.

How it works internally (x86-64 Linux, step by step):

1. The program puts the system-call number in a register (e.g. `rax`) and arguments in others (`rdi`, `rsi`, `rdx`…).
2. It executes the `syscall` instruction. The CPU switches to kernel mode and jumps to a fixed kernel entry point (the system-call handler).
3. The kernel looks up the number in the **system-call table**, validates the arguments, and runs the corresponding kernel function.

4. The result is placed in a register; the kernel executes a return-from-kernel instruction that switches back to user mode and resumes the program right after the `syscall`.

A RAW SYSTEM CALL VS. THE LIBRARY WRAPPER (LINUX, C)

```c
/* The C library gives you a friendly wrapper... */
ssize_t n = write(1, "hi\n", 3);   // fd 1 = stdout, 3 bytes

/* ...which internally does roughly this (conceptually): */
//   mov rax, 1        ; syscall number for write on x86-64
//   mov rdi, 1        ; arg1: file descriptor
//   mov rsi, buf      ; arg2: pointer to data
//   mov rdx, 3        ; arg3: byte count
//   syscall           ; <-- traps into the kernel
```

▣ KEY CONCEPT — API VS SYSTEM CALL

An **API** (Application Programming Interface) is the set of functions your code calls, such as the C library's `fopen()`, `printf()`, `malloc()`. A **system call** is the kernel entry underneath, such as `open`, `write`, `mmap`. Many API functions wrap one or more system calls — but not always: `strlen()` makes zero system calls, and one `printf()` may buffer many prints into a single `write`. The API is what you program against; the system call is what actually crosses into the kernel.

Aspect	API (library function)	System call
Lives in	User space (e.g. libc)	Kernel (entered via a trap)
Cost	Cheap — an ordinary function call	Expensive — mode switch + kernel work
Example	`printf`, `fopen`, `malloc`	`write`, `open`, `brk`/`mmap`
Portability	Portable across OSes (same C API)	OS-specific numbers & semantics
Purpose	Convenience, buffering, portability	The actual privileged service

Categories of system calls. Process control (`fork`, `execve`, `exit`, `wait`), file management (`open`, `read`, `write`, `close`), device management (`ioctl`), information (`getpid`, `time`), communication (`pipe`, `socket`, `shmget`), and protection (`chmod`, `setuid`).

⚑ INTERVIEW TIP

"Why are system calls expensive?" Because each one costs a **mode switch** (save user state, switch to kernel mode, run the handler, restore state) plus the kernel's own validation and work, and it can pollute CPU caches and pipelines. That is why libraries buffer — one big `write` beats a thousand tiny ones. This is also why high-performance systems batch I/O (`writev`, `io_uring`) to amortize the crossing.

> ⚑ **REAL WORLD**
>
> Run `strace ls` on Linux. You will see the actual stream of system calls `ls` makes — `openat`, `read`, `write`, `close`, `mmap`. This is the single best way to see the user/kernel boundary in action. We use it throughout Part XIV.

1.8 Traps, Interrupts, and Exceptions

All three are ways the CPU stops what it is doing and jumps into the kernel. Interviewers love the distinction because candidates blur it. Here is the clean version:

Term	Source	Timing	Meaning & example
Interrupt	External hardware (device)	Asynchronous	A device signals it needs attention, unrelated to the current instruction. E.g. keyboard press, packet arrival, disk finished, timer tick.
Exception / Fault	The running instruction (CPU)	Synchronous	The current instruction did something the CPU can't complete. E.g. divide-by-zero, invalid memory access, **page fault**.
Trap	A deliberate instruction	Synchronous	The program intentionally asks to enter the kernel. E.g. a **system call**, a debugger breakpoint. (A "software interrupt.")

> ▣ **KEY CONCEPT**
>
> **Asynchronous vs synchronous** is the crisp dividing line. An **interrupt** is asynchronous — it comes from the outside world and can arrive between any two instructions. An **exception** and a **trap** are synchronous — they are caused by the instruction currently executing. A trap is the intentional kind of exception ("I meant to do that"); a fault is the unintended kind ("something went wrong").

How the kernel responds. The CPU consults the **interrupt vector table** (a table of handler addresses indexed by interrupt/exception number), saves the current state, switches to kernel mode, and runs the matching **interrupt/exception handler**. When the handler finishes, the CPU restores state and resumes — either the interrupted program (for an interrupt) or, for a fault it fixed (like a page fault), re-runs the faulting instruction.

1.9 OS Services

Everything the OS offers falls into a handful of service categories. You have now met the mechanisms; these are the user-facing results.

- **Program execution** — load, run, and end programs (Part II).
- **I/O operations** — read/write files and devices without touching hardware (Part X).

- **File-system manipulation** — create, delete, search, and protect files (Part IX).
- **Communication** — let processes exchange data locally or over a network (Parts XI, XIII).
- **Error detection** — catch hardware and software faults and respond safely.
- **Resource allocation** — share CPU, memory, and devices among many programs (Parts IV, VII).
- **Accounting & logging** — track who used what.
- **Protection & security** — isolate programs and users (§1.10).

1.10 Protection and Security

These words are related but not identical, and mixing them up is a classic slip.

Protection

Controlling access by programs and users to the resources inside the system — e.g. stopping process A from reading process B's memory, or a user from deleting another user's files. It guards against **accidents and internal misuse**.

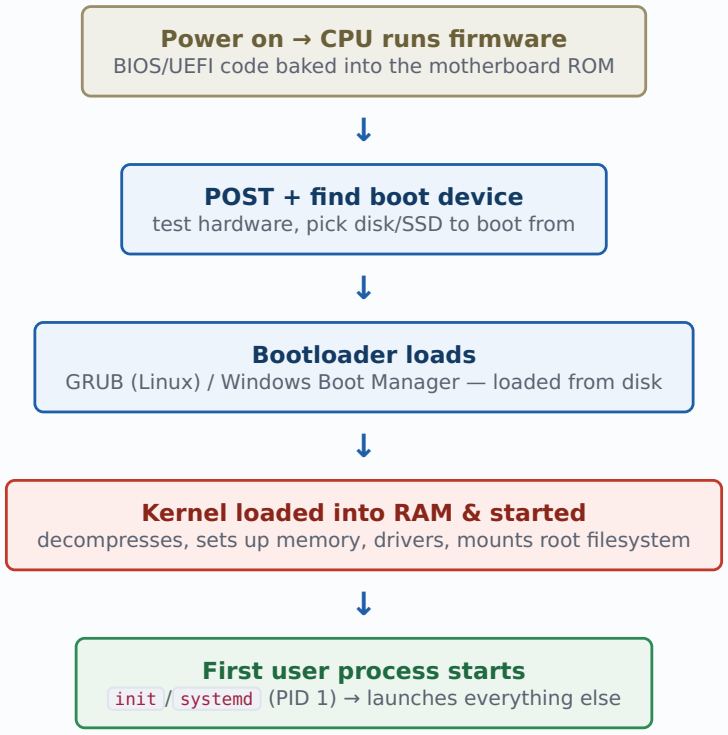
Security

Defending the system against external threats — malware, network attackers, stolen credentials. It guards against **deliberate, often external, attacks**.

Mechanisms include dual-mode operation, memory protection via the MMU (Part VII), user/group permissions (Part IX), and modern additions like ASLR and namespaces (Part XV). Protection is the foundation; security is built on top of it.

1.11 The Boot Process

The chicken-and-egg problem: the OS manages the hardware, but the OS is a program that must itself be loaded and run by the hardware. **Booting** (from "bootstrapping" — pulling yourself up by your bootstraps) is how the machine climbs from bare power-on to a running OS.



Cold boot to login. Each stage loads and hands control to the next, larger stage.

The detail-by-detail version — including what the firmware, bootloader, and PID 1 each do — is in Part XVI, "What happens when I turn on my computer?" For now, the shape is enough: firmware → bootloader → kernel → init → your login.

1.12 OS Architectures: Monolithic, Micro, Hybrid, Modular

Where should all this OS code live — one giant privileged blob, or many small pieces? This design choice defines an OS's **architecture**, and it is a favorite interview and exam topic.

Monolithic Kernel

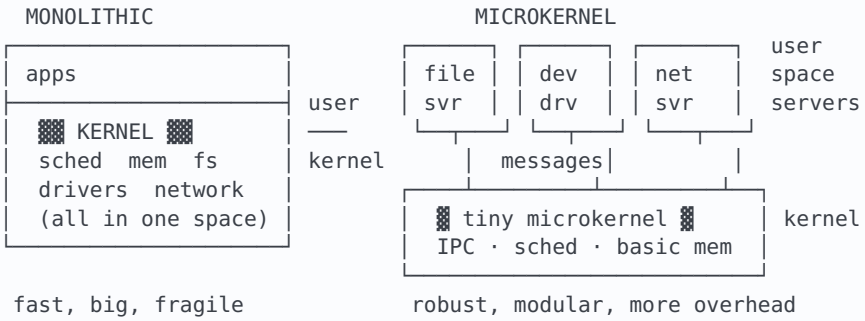
The entire OS — scheduling, memory, file systems, drivers, network stack — runs together in kernel space as one large program. Components call each other directly as fast function calls.

+ Very fast (no crossing between components). **−** Huge, complex; a bug in any driver can crash the whole kernel; hard to maintain in isolation. **Examples:** classic UNIX, Linux (though modular), BSD.

Microkernel

Keep the kernel tiny: only the bare essentials (scheduling, basic memory, and inter-process messaging) run in kernel mode. Everything else — file systems, drivers, networking — runs as ordinary **user-space servers** that talk via message passing.

+ Robust and secure (a crashed driver doesn't take down the kernel; just restart the server); modular; easier to verify. **−** Slower — lots of message passing and mode switches. **Examples:** Minix, QNX, seL4, the Mach core.



Monolithic puts everything in kernel space; microkernel pushes services out to user space.

Hybrid Kernel

A pragmatic middle ground: a mostly-monolithic kernel for speed, but structured with some microkernel ideas and message passing. **Examples:** Windows NT, macOS (XNU = Mach + BSD).

Modular Kernel

A monolithic core that can **load and unload components at runtime** as **modules** — you get monolithic speed with some of the flexibility of a microkernel. **Example:** Linux, via **loadable kernel modules** (a driver you `insmod` / `modprobe` without recompiling the kernel; see Part XV). Modern Linux is best described as a modular monolithic kernel.

Property	Monolithic	Microkernel	Hybrid	Modular
Where services run	Kernel space	User space	Mostly kernel	Kernel (loadable)
Speed	Highest	Lower	High	High
Fault isolation	Poor	Excellent	Medium	Poor–Medium
Extensibility	Hard	Easy	Medium	Easy (modules)
Example	Classic UNIX	QNX, seL4	Windows, macOS	Linux

⚠ **COMMON MISTAKE**

Saying "Linux is a pure monolithic kernel." It is monolithic in that services run in kernel space, but it is modular — drivers and features load and unload at runtime as kernel modules. The precise answer is "modular monolithic," and saying so signals real understanding.

✓ Check Your Understanding — Part I

CONCEPTUAL (EXPLAIN IN YOUR OWN WORDS)

1. State the two fundamental jobs of an operating system and give an example of each.
2. What is the difference between user space/kernel space and user mode/kernel mode?
3. Why can't a user program simply switch itself into kernel mode?
4. Distinguish an interrupt, an exception, and a trap by source and timing.
5. Explain the difference between an API and a system call.

APPLICATION

1. You call `printf("hi")` a thousand times in a loop. Explain why this may result in far fewer than a thousand `write` system calls.
2. A driver has a bug that dereferences a bad pointer. Predict the outcome on a monolithic kernel vs. a microkernel, and why.
3. Order these boot stages: init/systemd, bootloader, firmware (UEFI), kernel.

INTERVIEW-LEVEL

1. "Why are system calls expensive, and how do real systems reduce that cost?"
2. "Is Linux monolithic or a microkernel? Defend your answer."

CAN YOU EXPLAIN THIS? (NO NOTES)

The user/kernel boundary and the three ways to cross it · API vs system call · resource manager vs extended machine · monolithic vs microkernel trade-offs.

⚑ **PRACTICAL TASK**

On any Linux machine (or WSL), run `strace -c ls`. Read the summary: which system calls did `ls` make, and how many times? Identify one call that reads directory data and one that writes your terminal. You have just watched the abstraction stack work.

Mastered when... you can draw the Application → Library → System Call → Kernel → Hardware stack from memory, explain what enforces each boundary, and describe exactly what happens (in hardware and software) during a single `write()` call.

PART II

Processes

A program is a dead file on disk. A process is that program brought to life — given memory, a CPU, and an identity. Understanding the process is understanding the single most important object the operating system manages.

In this part: Program vs process · The process address space · The PCB · Five process states & transitions · `fork()`, `exec()`, `wait()` · Parent/child trees · Zombies, orphans & daemons · Context switching · Scheduling queues · CPU vs I/O bursts · Process isolation

PART II

Processes

Everything a computer does for you happens inside a process. When you open a browser, compile code, or run a server, the OS creates processes, feeds them CPU time, gives each its own protected memory, and cleans up after them. Master the process and half of operating systems falls into place.

2.1 Program vs Process

What is it? A **program** is a passive, static file on disk — a sequence of instructions and data (e.g. `/usr/bin/ls`). A **process** is a program in execution: an active entity with its own memory, CPU registers, a program counter, open files, and a unique identity.

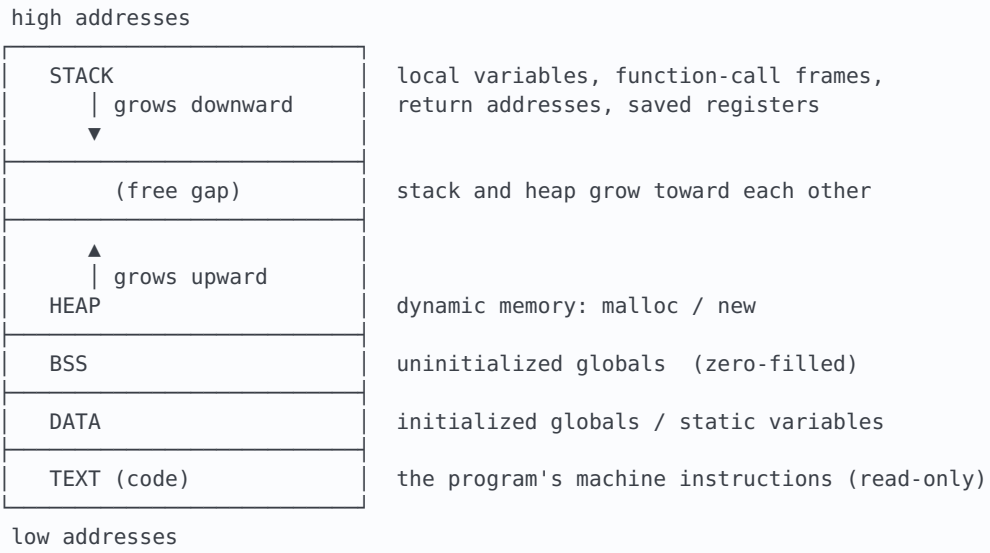
Simple intuition. A program is a recipe in a cookbook. A process is you actually cooking it — using a kitchen (memory), your hands (CPU), and a bookmark showing which step you are on (program counter). The same recipe can be cooked by two people at once in two kitchens: one program, two independent processes.

> ▣ **KEY CONCEPT**
>
> **Program : Process = Recipe : Cooking.** One program can spawn many processes (open your terminal twice — two `bash` processes from one `bash` program). A process is a program plus its execution state: memory image, registers, PC, kernel bookkeeping.

Aspect	Program	Process
Nature	Passive / static	Active / dynamic
Location	On disk (a file)	In memory (RAM), executing
Lifetime	Exists until deleted	Exists from creation to termination
Resources	None (just bytes)	CPU time, memory, open files, PID
Count	One file	Many processes from the same program

2.2 The Process Address Space

When the OS launches a process, it gives it a private **virtual address space** — a fresh, protected view of memory the process believes is entirely its own. That space is organized into segments. Knowing this layout explains stack overflows, `malloc`, and countless bugs.



A process's virtual address space. Text/data come from the program file; heap and stack grow at run time.

Text

The machine code. Read-only (so it can be shared between processes running the same program) and protected from accidental writes.

Data / BSS

Global and static variables. Data holds initialized ones; BSS holds zero-initialized ones (stored compactly as "reserve N zero bytes").

Heap

Dynamically allocated memory (`malloc` / `new`). Grows upward as you allocate; managed by the allocator on top of the `brk` / `mmap` syscalls.

Stack

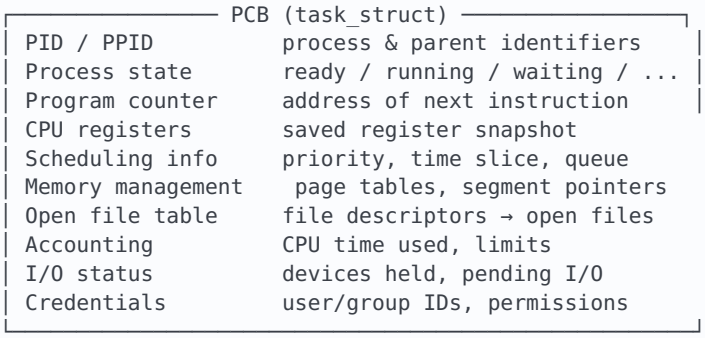
Function call frames: local variables, arguments, return addresses. Grows downward; automatically freed as functions return.

> ⚠ **COMMON MISTAKE**
>
> Confusing **stack** and **heap**. Locals live on the stack and vanish when the function returns — returning a pointer to a local is a bug. `malloc`'d memory lives on the heap and survives until you `free` it — forgetting to free is a memory leak.

2.3 The Process Control Block (PCB)

What is it? The **Process Control Block** is the kernel's data structure that stores everything the OS needs to know about a process. One PCB exists per process; it is the process, as far as the kernel is concerned. (In Linux the PCB is `struct task_struct`.)

Why do we need it? The CPU can run only one process at a time per core. To pause process A and later resume it exactly where it left off, the OS must save A's complete state somewhere — that somewhere is the PCB.



The PCB: the kernel's complete record of one process. Saving/restoring it is the essence of a context switch.

> ▣ **KEY CONCEPT**
>
> The PCB is the anchor for three big ideas at once: **context switching** (save/restore its registers & PC), **scheduling** (its state and priority determine what runs next), and **isolation** (its memory-management fields point to a private address space). The kernel keeps all PCBs in lists/queues so it can find and manage every process.

2.4 Process States and Transitions

A process is not always running — most of the time it is waiting for something (the CPU, or an I/O device). The OS models this with a small set of **states**. The five-state model is the one to know cold.

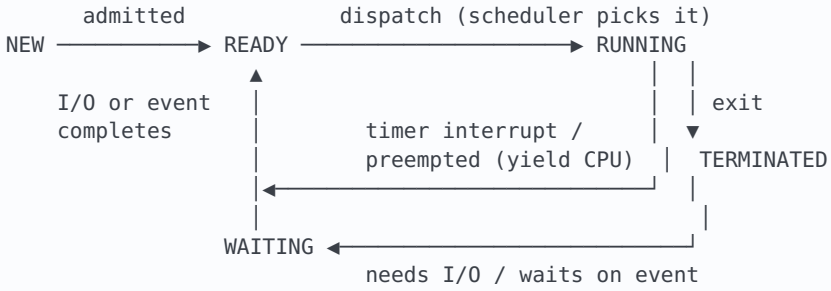
New
: The process is being created (PCB allocated, memory being set up).

Ready
: Loaded and willing to run — just waiting for the CPU. Sits in the **ready queue**.

Running
: Currently executing on a CPU. Only one process per core is here at any instant.

Waiting / Blocked
: Cannot proceed until some event happens (disk read completes, a lock is released, input arrives). Not competing for the CPU.

Terminated
: Finished executing; the OS is reclaiming its resources.



The five-state process model. Transitions are driven by the scheduler, timer interrupts, and I/O events.

The key transitions:

- **Ready → Running** (dispatch): the scheduler selects the process and gives it the CPU.
- **Running → Ready** (preemption): its time slice expired, or a higher-priority process arrived. It is still runnable — just paused.
- **Running → Waiting** (block): it requested I/O or a resource that is not ready. It voluntarily gives up the CPU.
- **Waiting → Ready** (wakeup): the awaited event happened; it rejoins the ready queue (note: it does not go straight to Running).
- **Running → Terminated**: it called `exit()` or was killed.

> ⚠ **COMMON MISTAKE**
>
> Thinking a process goes Waiting → Running directly when its I/O finishes. It does not — it goes Waiting → **Ready**, then must be scheduled before it runs. The CPU might be busy with someone else. This distinction matters in scheduling questions.

> ⚑ **INTERVIEW TIP**
>
> Draw the five-state diagram and name every transition and its cause. Then mention that real kernels add states: Linux has `TASK_RUNNING` (covers ready+running), `TASK_INTERRUPTIBLE` / `UNINTERRUPTIBLE` (two flavors of waiting), and `TASK_ZOMBIE`. Naming real kernel states signals depth.

2.5 Process Creation with fork()

What is it? On UNIX/Linux, new processes are born by **cloning**. The `fork()` system call creates a near-exact copy of the calling process. The original is the **parent**; the copy is the **child**. This builds a **process tree** rooted at PID 1 (`init` / `systemd`).

The famous trick — one call, two returns. `fork()` is called once but returns twice: once in the parent (returning the child's PID) and once in the child (returning 0). That return value is how each copy knows who it is.

FORK() — ONE CALL, TWO PROCESSES

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();              // clone this process here

    if (pid < 0) {
        perror("fork failed");   // no child was created
    } else if (pid == 0) {
        printf("Child : my PID=%d, parent=%d\n", getpid(), getppid());
    } else {
        printf("Parent: my PID=%d, child=%d\n", getpid(), pid);
        wait(NULL);                 // wait for the child to finish
    }
    return 0;
}
```

How it works internally. The kernel allocates a new PCB, copies the parent's state (open files, credentials, and a logical copy of its address space), assigns a new PID, and marks the child Ready. Copying all of memory would be wasteful, so modern kernels use **copy-on-write (COW)**: parent and child share the same physical pages, marked read-only; only when one writes does the kernel duplicate that page. This makes `fork()` cheap. (COW returns in Part VIII.)

⚠ **COMMON MISTAKE**

Assuming the parent runs before the child (or vice versa). After `fork()`, both are Ready and the scheduler decides the order — it is nondeterministic. Never write code that depends on which runs first; use `wait()` or synchronization if order matters.

2.6 Running a New Program with exec()

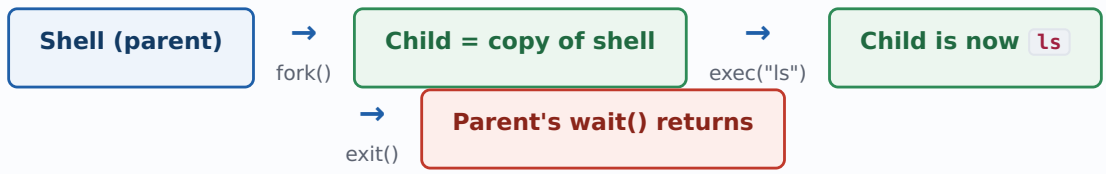
`fork()` gives you a copy of the same program. To run a different program, the child calls a member of the `exec()` family (`execl`, `execvp`, `execve`…). `exec()` **replaces** the current process's memory image with a new program — same PID, brand-new text/data/heap/stack. It does not return on success (there is nothing to return to; the old program is gone).

▣ **KEY CONCEPT — THE FORK/EXEC PATTERN**

This is how every shell launches every command: **fork** to create a child, then **exec** in the child to become the desired program, while the parent **wait**s. Separating "create a process" (`fork`) from "run a program" (`exec`) is a deliberate, powerful UNIX design — between the two, the child can redirect I/O, close files, or set up pipes (Part XI).

FORK + EXEC + WAIT: HOW A SHELL RUNS "LS -L"

```c
pid_t pid = fork();
if (pid == 0) {                        // --- child ---
    execlp("ls", "ls", "-l", NULL);    // become 'ls'; never returns if OK
    perror("exec failed");             // only reached on error
    _exit(1);
} else {                               // --- parent (the shell) ---
    int status;
    waitpid(pid, &status, 0);          // block until child finishes
    printf("child exited with %d\n", WEXITSTATUS(status));
}
```



The universal fork → exec → wait lifecycle behind every shell command.

2.7 Termination and wait()

A process ends by calling `exit(status)` (voluntary) or by receiving a fatal signal like `SIGKILL` (involuntary). On termination the kernel closes the process's files, frees its memory, and keeps a small **exit status** around so the parent can learn how the child fared.

The parent collects that status with `wait()` / `waitpid()`. This is called **reaping**. Until the parent reaps it, the dead child lingers as a **zombie** (next section). `wait()` also lets a parent synchronize with a child — blocking until it finishes.

2.8 Zombies, Orphans, and Daemons

Three special process conditions that interviewers love and beginners confuse. Learn them as a set.

Zombie Process

What: a child that has terminated but whose parent has not yet `wait()`ed for it. It is dead but still occupies a PCB slot (holding its exit status). **Why it exists:** so the parent can still read the exit status after the child is gone. **Danger:** a parent that never reaps its children leaks PCB entries; enough zombies can exhaust the process table. **Fix:** the parent must `wait()`, or handle `SIGCHLD`.

CREATING (AND OBSERVING) A ZOMBIE

```
pid_t pid = fork();
if (pid == 0) {
    _exit(0);              // child dies immediately
} else {
    sleep(30);             // parent naps WITHOUT wait() -> child is a zombie
    // run 'ps aux | grep defunct' now: you'll see it as <defunct>
}
```

Orphan Process

What: a child whose parent terminated first, leaving the child still running. **Who adopts it:** the kernel **reparents** orphans to `init`/`systemd` (PID 1), which dutifully reaps them when they finish. So orphans are harmless — they always get a responsible foster parent.

Daemon Process

What: a long-running background service with no controlling terminal — web servers, `sshd`, `cron`. **How it is made:** a process `fork`s, the parent exits (so the child becomes an orphan adopted by PID 1), and the child calls `setsid()` to start a new session, detaching from any terminal. The trailing **d** in names like `sshd`, `httpd`, `systemd` means "daemon."

Type	State	Parent	Problem?
Zombie	Dead, not yet reaped	Alive but hasn't waited	Yes — leaks PCB slots
Orphan	Still running	Dead; adopted by PID 1	No — PID 1 reaps it
Daemon	Running (background)	PID 1 (intentionally)	No — by design

⚠ **COMMON MISTAKE**

"You kill a zombie with `kill -9`." You cannot — it is already dead; there is nothing to kill. A zombie disappears when its parent reaps it. If the parent won't, kill the parent; the zombie is then reparented to PID 1, which reaps it instantly.

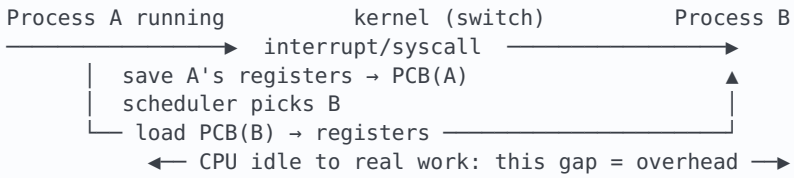
2.9 Context Switching

What is it? A **context switch** is the act of saving the state of the currently running process and restoring the state of another, so the CPU can switch from running process A to process B. It is the mechanism that makes one CPU appear to run many processes at once.

Simple intuition. A single receptionist (CPU) serving many customers (processes). To switch customers, the receptionist writes down exactly where they left off with customer A (save context), pulls out customer B's file (restore context), and continues. The note-taking itself takes time and serves no customer — that is **overhead**.

How it works internally. Triggered by a timer interrupt (preemption), a blocking system call, or a higher-priority wakeup, the kernel:

1. Saves the CPU registers and program counter of A into A's PCB.
2. Updates A's state (Running → Ready or Waiting) and picks B via the scheduler.
3. Loads B's registers, PC, and memory context (switches page tables / address space) from B's PCB.
4. Returns to user mode; B resumes exactly where it last stopped.



A context switch. The shaded gap does no useful application work — pure overhead.

> ⚑ **INTERVIEW TIP**
>
> "Why does context switching have overhead?" Two reasons: (1) the direct cost of saving/restoring registers and switching address spaces, and (2) the indirect cost — the new process starts with cold CPU caches and TLB (which may be flushed), so it runs slower until they warm up. This is why excessive switching (e.g. too many threads) hurts performance. Switching between threads of one process is cheaper: same address space, so no page-table switch or full TLB flush.

2.10 Scheduling Queues, CPU Bursts, and I/O Bursts

The OS keeps processes organized in queues as they move between states:

- **Job queue** — all processes in the system.
- **Ready queue** — processes in memory, Ready, waiting for the CPU.
- **Device / wait queues** — one per device; processes blocked on that device's I/O.

A process's life alternates between two kinds of activity:

CPU burst
: A stretch of computation using the CPU.

I/O burst
: A stretch of waiting for input/output (disk, network, user).

This **CPU–I/O burst cycle** is fundamental to scheduling (Part IV). **CPU-bound** processes (e.g. video encoding) have long CPU bursts; **I/O-bound** processes (e.g. a text editor) have short CPU bursts and lots of waiting. Good schedulers favor I/O-bound processes to keep devices busy.

2.11 Process Isolation

What & why. Each process gets its own virtual address space, so process A literally cannot see or corrupt process B's memory — a wild pointer in A crashes only A. This **isolation** is enforced by the hardware MMU (Part VII) under the kernel's control, and it is the reason one misbehaving app doesn't take down your whole machine.

Isolation is a double-edged sword: it keeps processes safe from each other, but it also means they cannot casually share data. To communicate, they need explicit, kernel-mediated mechanisms — which is exactly what **inter-process communication** provides.

2.12 Inter-Process Communication (Overview)

Because processes are isolated, the OS offers controlled channels for them to exchange data. There are two broad families:

- **Shared memory** — two processes map the same region of physical memory; fastest, but they must synchronize access themselves.
- **Message passing** — processes send/receive discrete messages via the kernel (pipes, message queues, sockets); safer and simpler, but involves copying and system calls.

The full toolkit — pipes, named pipes, message queues, shared memory, signals, sockets, and RPC — with working C examples is **Part XI**. For now, just hold the idea: isolation is the default; IPC is how you deliberately poke holes in it.

✓ Check Your Understanding — Part II

CONCEPTUAL

1. Give the recipe/cooking analogy for program vs process and state what a process has that a program lacks.
2. Name the four main segments of a process address space and what each holds.
3. List the five process states and the cause of every transition between them.
4. What is stored in a PCB, and why is it central to context switching?
5. Define zombie, orphan, and daemon in one sentence each.

APPLICATION

1. A `fork()`'d program prints a line before and after the fork. If the line before says "start" and the line after says "hi", how many total lines are printed and why?
2. Your server spawns worker children but never calls `wait()`. After a day it can't create new processes. Diagnose the bug and give two fixes.
3. Explain why switching between two threads of one process is cheaper than switching between two processes.

INTERVIEW-LEVEL

1. "Why does `fork()` return twice, and how does copy-on-write make it cheap?"
2. "Walk me through exactly what happens, step by step, when a shell runs `grep foo file`."

CAN YOU EXPLAIN THIS? (NO NOTES)

The five-state diagram with every transition · the fork/exec/wait pattern · what a context switch saves and restores (and why it has overhead) · zombie vs orphan.

⚑ PRACTICAL TASK

Compile the `fork()` example. Then write a version where the child `_exit`s and the parent `sleep`s for 20 s without `wait()`. While it sleeps, run `ps -el | grep defunct` in another terminal and find your zombie. Then add `wait(NULL)` and confirm the zombie is gone.

Mastered when... you can write a fork/exec/wait program from scratch, predict its output, explain the process tree it creates, and describe what the kernel does to its PCBs at each step.

PART III

Threads & Concurrency

A process gives you isolation. A thread gives you a way to do several things at once inside one process, sharing its memory. That sharing is the source of both a thread's power and every concurrency bug you will ever chase.

In this part: What a thread is · Process vs thread · What's shared vs private · Why threads · User-level vs kernel-level threads · Many-to-one, one-to-one, many-to-many · The TCB · Concurrency vs parallelism · Multi-core · Race conditions · Critical sections · Thread safety

PART III

Threads & Concurrency

If a process is a running program with its own memory, a thread is a single flow of execution within that memory. Threads let one program keep a UI responsive while it downloads a file, or serve thousands of users at once — but because they share memory, they force us to confront concurrency head-on.

3.1 What Is a Thread?

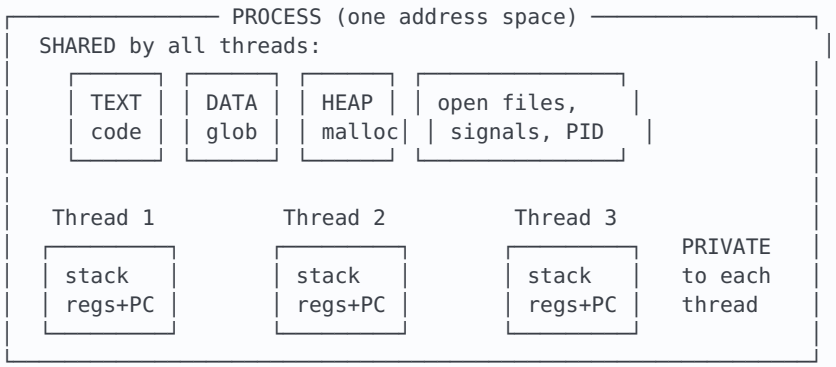
What is it? A **thread** (short for "thread of execution") is the smallest unit the CPU schedules — a single sequential flow of instructions with its own program counter, registers, and stack, running inside a process and sharing that process's memory and resources with its sibling threads.

Simple intuition. A process is a kitchen; a thread is a cook. One cook (single-threaded) does everything in sequence. Add more cooks (threads) in the same kitchen and they work simultaneously, sharing the same ingredients, counters, and fridge (memory). They get more done — but if two cooks grab the same knife at once, you have a problem (a race condition).

Why do we need it? Before threads, doing two things at once meant two processes, which are expensive to create and cannot easily share data. Threads are lightweight and share memory by default, so a single program can naturally overlap work — compute while waiting on I/O, or spread computation across CPU cores.

3.2 Process vs Thread — What's Shared, What's Private

This is the most-asked threading question. The answer hinges on one picture: threads of a process share almost everything, but each keeps its own execution state.



Threads share the process's code, globals, heap, and OS resources; each has its own stack, registers, and program counter.

SHARED across threads of a process	PRIVATE to each thread
Code (text segment)	Stack (local variables, call frames)
Global & static data	CPU registers
Heap (`malloc`'d memory)	Program counter (PC)
Open file descriptors, sockets	Thread ID (TID)
Process ID, signal handlers, working dir	Errno, thread-local storage (TLS)

▣ **KEY CONCEPT**

Threads share **memory** (so they can cooperate cheaply — no copying) but keep their own **stack + registers + PC** (so each can be at a different point in the code). Sharing memory is why threads are fast and why they need synchronization: two threads writing the same global with no lock is a data race.

Aspect	Process	Thread
Memory	Own private address space	Shares the process's address space
Creation cost	Heavy (new address space, COW)	Light (just a stack + PCB-like TCB)
Context switch	Expensive (switch page tables, flush TLB)	Cheaper (same address space)
Communication	Needs IPC (pipes, shared mem)	Shared variables (direct)
Isolation / safety	Strong — a crash stays contained	Weak — one thread's bad write can corrupt all
One crashes	Others unaffected	Can take down the whole process

⚠ **COMMON MISTAKE**

"Threads have their own heap." No — the **heap is shared**. If thread A `malloc`s and passes the pointer to thread B, B can use it. Only the **stack** is per-thread. This is precisely why passing a pointer to a local (stack) variable of one thread to another thread is dangerous.

3.3 Why Threads? (And When Not To)

- **Responsiveness** — one thread handles the UI while another does slow work; the app never freezes.
- **Resource sharing** — threads share memory for free; no IPC plumbing needed.
- **Economy** — creating and switching threads is far cheaper than processes.
- **Scalability / parallelism** — on a multi-core CPU, threads run truly simultaneously, one per core, cutting wall-clock time.

The cost: shared memory means shared bugs. Race conditions, deadlocks, and hard-to-reproduce "Heisenbugs" are the price. And more threads is not always faster — see the tip below.

> ⚑ **INTERVIEW TIP**
>
> "Why can too many threads reduce performance?" Because (1) each thread adds context-switching and scheduling overhead; (2) beyond the core count, threads compete for the same cores, so you pay switching cost without gaining parallelism; (3) more threads means more lock contention (they serialize on shared locks) and more cache thrashing. The sweet spot for CPU-bound work is roughly one thread per core; for I/O-bound work you can use more, since threads spend time blocked.

3.4 User-Level vs Kernel-Level Threads

Who manages the threads — a user-space library, or the kernel? This choice has real consequences.

User-level threads (ULT)
: Managed entirely by a library in user space; the kernel sees only one process and knows nothing of the threads. **+** Very fast to create/switch (no system call); portable. **−** If one thread makes a blocking system call, the whole process blocks (the kernel blocks the one process it can see); cannot use multiple cores.

Kernel-level threads (KLT)
: Managed by the kernel, which schedules each thread individually. **+** One thread blocking doesn't stop the others; real parallelism across cores. **−** Creation and switching involve the kernel, so they're slower than ULTs.

Modern systems (Linux, Windows) use kernel-level threads — every `pthread` maps to a kernel-scheduled thread — because true multi-core parallelism and non-blocking behavior matter more than the last drop of switch speed.

3.5 Multithreading Models

These describe how user threads map onto kernel threads.

Many-to-One

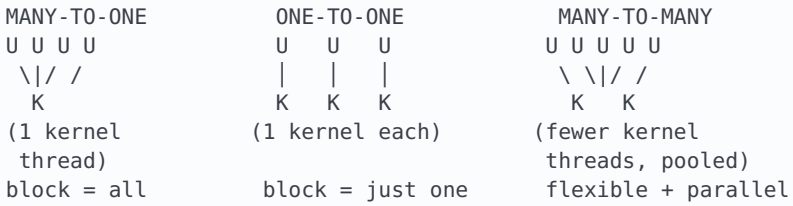
Many user threads map to a single kernel thread. Thread management is cheap and in user space, but one blocking call blocks everyone, and no multi-core parallelism. (Old "green threads.")

One-to-One

Each user thread maps to its own kernel thread. True concurrency and parallelism; a block affects only that thread. Cost: creating many threads creates many kernel threads. This is what **Linux and Windows use today**.

Many-to-Many

Many user threads multiplexed onto a smaller-or-equal pool of kernel threads. Combines flexibility (create as many user threads as you like) with parallelism (multiple kernel threads). Complex to implement; used by some runtimes (e.g. Go's goroutine scheduler is a modern many-to-many-style design).



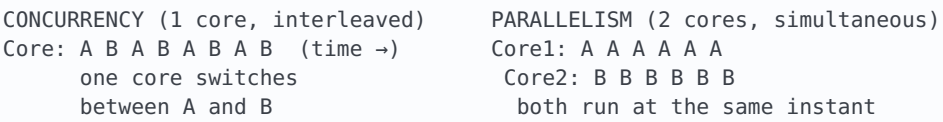
U = user thread, K = kernel thread. The mapping decides blocking behavior and parallelism.

Thread Control Block (TCB). Just as a process has a PCB, each thread has a lighter **TCB** holding its thread ID, register set, program counter, stack pointer, and scheduling state. Threads of one process share the PCB's memory and resource fields but each have their own TCB.

3.6 Concurrency vs Parallelism

These two words are used interchangeably in casual speech and must be distinguished precisely in an interview.

> ▣ **KEY CONCEPT**
>
> **Concurrency** = dealing with many things at once (structure): tasks are in progress over overlapping time periods, possibly interleaved on one core. **Parallelism** = doing many things at the exact same instant (execution): tasks literally run simultaneously on multiple cores. Concurrency is about how you structure work; parallelism is about how it executes. You can have concurrency on a single core (via interleaving); you need multiple cores for parallelism.



Concurrency interleaves; parallelism overlaps in real time. Parallel work is concurrent, but concurrent work need not be parallel.

Analogy. A single barista taking one order while steaming milk for another, alternating between them, is concurrent. Two baristas each making a full drink at the same time is parallel.

3.7 Multi-Core Systems

A modern CPU packs several **cores**, each a full processing unit able to run a thread independently. The OS scheduler spreads runnable threads across cores, achieving real parallelism. Some cores support **simultaneous multithreading** (Intel's Hyper-Threading), presenting two logical CPUs per physical core to better utilize execution units. To software, each logical CPU is just another place to run a thread — which is why one-to-one threading maps so naturally onto modern hardware.

3.8 Race Conditions and Critical Sections

The moment two threads share mutable data, correctness is at risk. This section introduces the problem; Part V solves it in full.

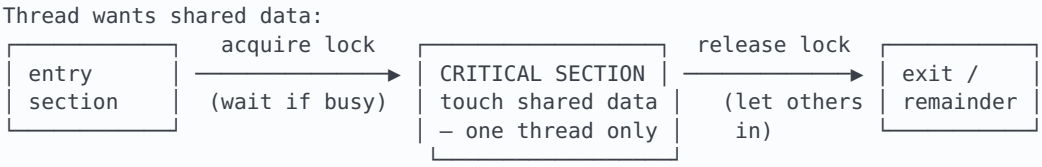
Race condition. A **race condition** occurs when the outcome of a program depends on the timing or interleaving of threads accessing shared data — and at least one is writing. The result is nondeterministic: it might work a thousand times and corrupt data on the thousand-and-first.

THE CLASSIC RACE: TWO THREADS INCREMENTING A SHARED COUNTER

```
// counter++ is NOT atomic. The CPU actually does three steps:
//   1) read  counter  into a register
//   2) add   1
//   3) write register back to counter

// If two threads interleave those steps on counter = 5:
//   T1 reads 5 ─┐                    T2 reads 5 ─┐
//   T1 adds → 6 │                    T2 adds → 6 │
//   T1 writes 6 ┘                    T2 writes 6 ┘
// Two increments happened, but counter is 6, not 7. One update was LOST.
```

Critical section. A **critical section** is a piece of code that accesses shared data and must not be executed by more than one thread at a time. The fix for a race is to enforce **mutual exclusion** over the critical section — only one thread inside at once — using a lock, mutex, or semaphore (Part V).



Guarding a critical section with a lock: enter one at a time, do the shared work, then release.

3.9 Thread Safety

Code is **thread-safe** if it behaves correctly when called from multiple threads at once, regardless of interleaving. You achieve it by (a) avoiding shared mutable state (use locals/immutable data), (b) protecting shared state with locks, or (c) using atomic operations. A function that keeps hidden shared state (like the classic `strtok`) is not thread-safe; reentrant variants (`strtok_r`) are provided precisely for threaded use.

A MINIMAL POSIX THREADS (PTHREADS) PROGRAM

```
#include <pthread.h>
#include <stdio.h>

void *worker(void *arg) {
    long id = (long)arg;
    printf("thread %ld running\n", id);
    return NULL;
}

int main(void) {
    pthread_t t[4];
    for (long i = 0; i < 4; i++)
        pthread_create(&t[i], NULL, worker, (void *)i);  // spawn
    for (int i = 0; i < 4; i++)
        pthread_join(t[i], NULL);                        // wait for each
    return 0;                 // compile with: gcc prog.c -lpthread
}
```

✓ Check Your Understanding — Part III

CONCEPTUAL

1. List exactly what threads of a process share and what each keeps private.
2. Give the precise difference between concurrency and parallelism, with an example of concurrency without parallelism.
3. Contrast user-level and kernel-level threads, especially blocking behavior.
4. Describe the three multithreading models and which one Linux uses.
5. Define a race condition and a critical section.

APPLICATION

1. Two threads each run `counter++` 1,000,000 times on a shared `counter` starting at 0, with no lock. What is the range of possible final values, and why?
2. You move a CPU-bound program from 4 threads to 40 threads on a 4-core machine and it gets slower. Explain.
3. Why can passing a pointer to a thread's local variable to another thread cause corruption?

INTERVIEW-LEVEL

1. "When would you choose multiple processes over multiple threads?"
2. "Is a single-core machine capable of concurrency? Of parallelism? Explain."

CAN YOU EXPLAIN THIS? (NO NOTES)

The shared-vs-private thread memory picture · concurrency vs parallelism · why `counter++` races · one-to-one vs many-to-one threading.

⚑ PRACTICAL TASK

Write a pthreads program where N threads each increment a shared global counter one million times with no lock. Print the final value. Run it several times — watch it come out different and wrong. Keep this program; in Part V you will fix it with a mutex and see it print the correct value every time.

Mastered when... you can draw the thread memory model, explain why shared mutable state races, and predict when adding threads helps versus hurts.

PART IV

CPU Scheduling

Many processes are Ready; only one CPU is free. Which one runs next? The scheduler answers that question thousands of times a second, and its policy decides whether your system feels snappy or sluggish, fair or starving.

In this part: Why scheduling · The metrics (turnaround, waiting, response, throughput) · Preemptive vs non-preemptive · FCFS · SJF · SRTF · Priority · Round Robin · Multilevel & multilevel-feedback queues · Starvation, aging, the convoy effect · Fully solved numerical problems

PART IV

CPU Scheduling

CPU scheduling is where operating-systems theory becomes arithmetic you can be tested on. This part teaches every classic algorithm the way an interviewer or examiner wants it: the intuition, the mechanism, a Gantt chart, and the full turnaround/waiting-time calculation worked to the last number.

4.1 Why CPU Scheduling? The Metrics

A process alternates CPU bursts with I/O bursts (Part II). While one process waits on I/O, the CPU would sit idle — a waste. The **scheduler** keeps the CPU busy by picking another Ready process to run. The **dispatcher** is the component that actually performs the context switch to hand the CPU over. The rules the scheduler follows are the **scheduling algorithm**, and we judge algorithms by these metrics:

Metric	Definition	Goal
Arrival Time (AT)	When the process enters the ready queue	— (given)
Burst Time (BT)	CPU time the process needs	— (given)
Completion Time (CT)	When the process finishes	— (computed)
Turnaround Time	TAT = CT − AT (total time in system)	Minimize
Waiting Time	WT = TAT − BT (time spent waiting in ready queue)	Minimize
Response Time	RT = (first time on CPU) − AT	Minimize
Throughput	Processes completed per unit time	Maximize
CPU Utilization	Fraction of time the CPU is busy	Maximize
Fairness	No process is unduly denied the CPU	Maximize

▣ **KEY CONCEPT — THE THREE FORMULAS THAT SOLVE EVERY PROBLEM**

Turnaround Time = Completion Time − Arrival Time

Waiting Time = Turnaround Time − Burst Time

Response Time = First-CPU Time − Arrival Time

Build the Gantt chart first, read off each Completion Time, then apply these. Every numerical in this part — and in your exam — is just these three lines applied carefully.

Two problems to watch for

Starvation

A process waits indefinitely because others keep being preferred (common with priority and SJF, where a low-priority or long job may never get picked).

Convoy effect

Short processes stuck waiting behind one long process (classic FCFS problem), tanking average waiting time — like a line of cars stuck behind one slow truck.

4.2 Preemptive vs Non-Preemptive

Aspect	Non-Preemptive	Preemptive
CPU taken away?	No — a process runs until it finishes or blocks	Yes — the OS can forcibly pause a running process
Triggered by	Process exits or requests I/O	Timer interrupt, higher-priority arrival
Responsiveness	Poorer (long job blocks others)	Better (can react to new/urgent work)
Overhead	Lower (fewer switches)	Higher (more context switches)
Examples	FCFS, SJF, non-preemptive Priority	SRTF, Round Robin, preemptive Priority

Modern general-purpose OSes are **preemptive** — essential for interactivity and to stop any one process from hogging the CPU.

4.3 FCFS — First-Come, First-Served

Intuition. A queue at a shop: serve whoever arrives first, to completion. **Working.** Non-preemptive; the ready queue is a simple FIFO. Simplest possible policy.

Worked example. Four processes:

Process	Arrival	Burst
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Order of service = order of arrival: P1, P2, P3, P4.



Boundaries: P1 0→5, P2 5→8, P3 8→16, P4 16→22.

Process	AT	BT	CT	TAT = CT−AT	WT = TAT−BT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6
P4	3	6	22	19	13

Average TAT = (5+7+14+19)/4 = **11.25** Average WT = (0+4+6+13)/4 = **5.75**

+ Simple, fair in arrival order, no starvation. **−** The **convoy effect**: put a long job first and every short job waits behind it, ballooning average waiting time. Non-preemptive, so bad for interactivity.

> ⚠ **COMMON MISTAKE**
>
> Forgetting idle time. If no process has arrived yet, the CPU sits idle and the timeline still advances. E.g. if P1 arrives at t=3 with nothing before it, the Gantt starts with idle [0→3], and P1's completion is 3+BT, not BT.

4.4 SJF — Shortest Job First (Non-Preemptive)

Intuition. Always run the Ready process with the smallest CPU burst next. Clearing the quick jobs first drags the average waiting time down. **SJF is provably optimal** for minimum average waiting time — among non-preemptive algorithms, no policy beats it.

Worked example (all arrive at t=0 for clarity): P1 BT=6, P2 BT=8, P3 BT=7, P4 BT=3. Pick shortest first: P4(3), P1(6), P3(7), P2(8).



Boundaries: P4 0→3, P1 3→9, P3 9→16, P2 16→24.

Process	BT	CT	TAT	WT
P4	3	3	3	0
P1	6	9	9	3
P3	7	16	16	9
P2	8	24	24	16

Average WT = (0+3+9+16)/4 = **7.0** (FCFS on the same set gives 10.25 — SJF wins)

+ Optimal average waiting time. **−** Requires knowing burst times in advance (impossible in general — the OS must estimate them, e.g. exponential averaging of past bursts); risks **starvation** of long jobs if short jobs keep arriving.

4.5 SRTF — Shortest Remaining Time First (Preemptive SJF)

Intuition. The preemptive version of SJF: whenever a new process arrives, compare its burst against the remaining time of the running process; if the newcomer is shorter, preempt. Decisions are re-made at every arrival.

Worked example: P1(AT=0,BT=8), P2(AT=1,BT=4), P3(AT=2,BT=9), P4(AT=3,BT=5).

- t=0: only P1 → run P1.
- t=1: P2(4) arrives; P1 remaining=7. 4<7 → preempt, run P2.
- t=2,3: P3(9), P4(5) arrive; P2 remaining (3, then 2) still smallest → keep P2.
- t=5: P2 done. Remaining P1=7, P3=9, P4=5 → run P4 (5).
- t=10: P4 done. Remaining P1=7, P3=9 → run P1 (7).
- t=17: P1 done. Run P3 (9) → done at 26.



Boundaries: P1 0→1, P2 1→5, P4 5→10, P1 10→17, P3 17→26.

Process	AT	BT	CT	TAT	WT
P1	0	8	17	17	9
P2	1	4	5	4	0
P3	2	9	26	24	15
P4	3	5	10	7	2

Average WT = (9+0+15+2)/4 = **6.5** Average TAT = (17+4+24+7)/4 = **13.0**

+ Lowest possible average waiting time of all these algorithms. **−** Heavy preemption overhead; still needs burst estimates; worst starvation risk for long jobs.

4.6 Priority Scheduling

Intuition. Each process has a priority number; the CPU always goes to the highest-priority Ready process. (Convention here: smaller number = higher priority.) Can be preemptive or non-preemptive. SJF is actually priority scheduling with priority = 1/burst.

Worked example (non-preemptive, all AT=0): P1(BT=4,pri=3), P2(BT=3,pri=1), P3(BT=1,pri=4), P4(BT=5,pri=2). Order by priority: P2, P4, P1, P3.



Boundaries: P2 0→3, P4 3→8, P1 8→12, P3 12→13.

WT: P2=0, P4=3, P1=8, P3=12 → Average WT = 23/4 = **5.75**

▣ **KEY CONCEPT — STARVATION & AGING**

Priority scheduling can **starve** low-priority processes forever if high-priority ones keep arriving. The standard cure is **aging**: gradually raise the priority of a process the longer it waits, so every process eventually becomes high-priority enough to run. (Legend: when MIT's IBM 7094 was shut down in 1973, a low-priority job submitted in 1967 was found still waiting.)

4.7 Round Robin (RR)

Intuition. FCFS with a stopwatch. Each process gets a fixed **time quantum** (time slice); if it doesn't finish, it's preempted and sent to the back of the ready queue. Designed for fairness and good response time — the heart of interactive time-sharing.

Worked example (quantum q=2, all AT=0): P1 BT=5, P2 BT=4, P3 BT=2, P4 BT=1.



Boundaries: P1 0→2, P2 2→4, P3 4→6, P4 6→7, P1 7→9, P2 9→11, P1 11→12.

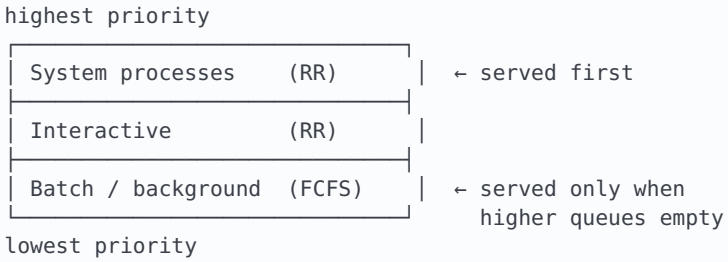
Process	BT	CT	TAT	WT	RT
P1	5	12	12	7	0
P2	4	11	11	7	2
P3	2	6	6	4	4
P4	1	7	7	6	6

Average WT = 24/4 = **6.0** Average TAT = 36/4 = **9.0** Average RT = 12/4 = **3.0**

> ⚑ **INTERVIEW TIP — CHOOSING THE QUANTUM**
>
> The time quantum is a knob. **Too large** → RR degenerates into FCFS (each job finishes in one slice), hurting response time. **Too small** → excessive context-switch overhead eats the CPU. Rule of thumb: the quantum should be large enough that ~80% of CPU bursts finish within it, typically 10–100 ms. RR optimizes response time and fairness, not average waiting time.

4.8 Multilevel Queue (MLQ)

Real systems have different classes of process (interactive, batch, system). MLQ splits the ready queue into several separate queues, each with its own scheduling algorithm, and processes are permanently assigned to one queue by type.



Multilevel queue: fixed classes, each with its own policy; higher queues preferred.

− Rigid — a process is stuck in its queue, and low queues can starve. That rigidity motivates the next algorithm.

4.9 Multilevel Feedback Queue (MLFQ)

The most sophisticated classic scheduler, and the closest to what real OSes use. Like MLQ, but processes can **move between queues** based on observed behavior:

- New processes enter the top (highest-priority, short-quantum) queue.
- A process that uses its whole quantum (CPU-bound) is **demoted** to a lower, longer-quantum queue.
- A process that yields early for I/O (interactive) stays high or is **promoted**.
- Periodic **aging** boosts long-waiting processes to prevent starvation.

▣ KEY CONCEPT

MLFQ learns each process's nature without being told its burst time: interactive jobs (short bursts) naturally float to the top for great responsiveness, while CPU hogs sink to the bottom where they run efficiently in long slices. It approximates SJF's benefits without needing to know burst times in advance. This is why it's the design basis for Windows, macOS, and older Linux schedulers.

4.10 Comparison & How to Choose

Algorithm	Preempt?	Optimizes	Starvation	Key weakness
FCFS	No	Simplicity	No	Convoy effect
SJF	No	Avg waiting time	Yes	Needs burst times
SRTF	Yes	Avg waiting time (best)	Yes	Overhead + estimates
Priority	Either	Importance	Yes	Starvation (use aging)
Round Robin	Yes	Response time, fairness	No	Quantum tuning
MLFQ	Yes	Balanced, adaptive	No (aging)	Complex to tune

Real-world: Linux's modern **CFS** (Completely Fair Scheduler) doesn't use fixed slices at all — it tracks each task's "virtual runtime" and always runs the one that has had the least CPU, approximating perfect fairness. The classic algorithms above remain the vocabulary for reasoning about and interviewing on scheduling.

✓ Check Your Understanding — Part IV

CONCEPTUAL

1. Write the formulas for turnaround, waiting, and response time.
2. What is the convoy effect and which algorithm suffers from it?
3. Why is SJF optimal for average waiting time, and why can't we use it directly?
4. Explain starvation and how aging fixes it.
5. How does MLFQ approximate SJF without knowing burst times?

APPLICATION (SOLVE FULLY — GANTT + TABLE)

1. P1(AT0,BT4), P2(AT1,BT3), P3(AT2,BT1), P4(AT3,BT2): compute average WT and TAT under **FCFS**, then under **SRTF**. Which is better?
2. Same four processes under **Round Robin, q=2**: give the Gantt chart and average response time.
3. Show that increasing RR's quantum toward infinity turns it into FCFS.

INTERVIEW-LEVEL

1. "Which scheduler would you pick for a real-time system vs. a batch server vs. a desktop, and why?"
2. "Design a scheduler that gives interactive apps good response time and batch jobs good throughput." (Hint: MLFQ.)

CAN YOU EXPLAIN THIS? (NO NOTES)

Build a Gantt chart and compute WT/TAT for any algorithm · convoy effect · the quantum trade-off in RR · why MLFQ adapts to process behavior.

⚑ **PRACTICAL TASK**

On Linux, run `nice -n 19 yes > /dev/null &` and `nice -n -5 yes > /dev/null &` (the second may need `sudo`), then watch them in `top`. Observe how the scheduler splits CPU by priority. Kill them with `kill %1 %2`. You've just driven the scheduler by hand (more in Part XIV).

Mastered when... given any process table you can produce the correct Gantt chart and average waiting/turnaround/response times for FCFS, SJF, SRTF, Priority, and Round Robin — and explain which metric each algorithm optimizes.